Exactly. We'll make the **language-specific consolidation round** the bridge between "I know data structures" and "I can actually program systems in C/Java/Python."

PART 1B — Language-Specific Consolidation

We'll take the same concepts from Levels 0–21 and progressively expose what is **fundamentally different** about C, Java, and Python.

TRACK A — C CONSOLIDATION

C1 — Pointers Revisited

Q177 — Array Through a Pointer

Given:

int arr[] = {10, 20, 30, 40, 50};

Print every element using:

1. Array indexing
2. Pointer arithmetic
3. A pointer variable
4. A function receiving a pointer

Then answer:

arr
&arr[0]
arr + 1
*(arr + 1)
&arr

What is the difference?

Q178 — Modify Through a Pointer

Create:

void increment(int *x)

so that:

x = 10

becomes:

11

Then:

void incrementArray(int *arr, int n)

increments every element.

Q179 — Pointer to Pointer

Create:

int **p;

and demonstrate:

x
↓
p
↓
address
↓
value

Then write a function capable of modifying a pointer itself.

This is preparation for linked lists and dynamic data structures.

C2 — Dynamic Memory

Q180 — Dynamic Array

Ask the user for N.

Allocate:

int *arr;

using:

malloc()

Read `N` integers and print them.

Then properly release the memory.

Q181 — `malloc` vs `calloc`

Create two arrays:

malloc()
calloc()

and inspect their initial contents.

Determine:

- What is initialized?
- What isn't?
- What arguments do they take?
- What do they return?

Q182 — Resize the Array

Start with capacity `5`.

When the array becomes full:

capacity *= 2

and use:

realloc()

to expand it.

Build:

DynamicArray

with:

push()
pop()
get()
set()
size()
capacity()
free()

This is your first miniature version of a **dynamic array library**.

C3 — Structures

Q183 — Student Structure

Create:

```
struct Student {
    char name[100];
    int age;
    float marks;
};
```

Create functions:

createStudent()
printStudent()
updateMarks()

Then create an array of students.

Q184 — Structure Pointer

Compare:

student.name

with:

studentPtr->name

Understand:

.
->

and when each is used.

C4 — Linked List Deep Dive

Q185

Implement:

createNode()
insertBeginning()
insertEnd()
insertAt()
deleteBeginning()
deleteEnd()
deleteValue()
search()
display()
destroy()

using dynamically allocated nodes.

Then deliberately test:

empty list
one node
two nodes
deleting head
deleting tail
deleting nonexistent value

C5 — Function Pointers

Now we reach one of the most important C-specific concepts.

Q186 — Basic Function Pointer

Create:

int add(int a, int b);
int subtract(int a, int b);

Create a function pointer:

int (*operation)(int, int);

and use it to call both functions.

Q187 — Calculator Through Function Pointer

Create:

calculate(a, b, operation)

where `operation` can point to:

add
subtract
multiply
divide

Now the **function itself becomes an argument**.

Q188 — Callback

Create:

processArray(array, size, callback)

where callback determines what happens to each element.

For example:

print(element)
square(element)
double(element)

This will later map directly to:

- Java interfaces/lambdas
- Python functions/lambdas

C6 — Generic Programming

Q189 — `void *`

Investigate:

void *

Build a generic-style function:

swap(void *a, void *b, size_t size)

Then use it with:

int
float
double

Investigate:

memcpy()
memmove()
memcmp()

and understand why `void *` is useful.

Q190 — Generic Sorting

Revisit:

qsort()

Create different comparison functions:

ascendingInt()
descendingInt()
absoluteValue()

Then investigate what `qsort()` is really doing with the function pointer.

TRACK B — JAVA CONSOLIDATION

Java C1 — Classes → OOD

Q191 — Student Class

Create:

class Student

with:

name
rollNumber
marks

Implement:

display()
calculateGrade()

Then make fields private.

Q192 — Constructor Exploration

Create:

default constructor
parameterized constructor
copy-style constructor

Investigate:

this

and constructor overloading.

Java C2 — Encapsulation

Q193

Make:

private fields
public getters
public setters

Then decide:

> Should every field have a setter?

Create an example where allowing unrestricted modification would be bad design.

Java C3 — Inheritance

Q194

Create:

Person
↓
Student
↓
ResearchStudent

Give each class its own behavior.

Explore:

super
this
@Override

Java C4 — Polymorphism

Q195

Create:

Animal
├── Dog
├── Cat
└── Bird

Each implements:

sound()

Store all of them in:

List<Animal>

and call:

sound()

without knowing the actual subtype.

This demonstrates runtime polymorphism.

Java C5 — Abstract Classes

Q196

Create:

abstract class Shape

with:

area()
perimeter()

Create:

Circle
Rectangle
Triangle

and implement the methods.

Java C6 — Interfaces

Q197

Create:

interface Printable

and:

interface Savable

Then create a class implementing both.

Investigate:

default methods
static interface methods
private interface methods

where supported by your Java version.

Java C7 — Composition

Q198

Create:

Car
 ├── Engine
 ├── Wheel
 └── Battery

The car **has** these objects.

Compare:

inheritance

with:

composition

and determine which relationship is appropriate.

Java C8 — Generics

Q199

Create:

class Box<T>

supporting:

set()
get()

Use:

Box<Integer>
Box<String>
Box<Student>

Then understand why generics are preferable to using `Object` in many cases.

Java C9 — Collections Framework

Revisit every data structure.

Q200

Implement your own:

Stack
Queue
Map
Set

Then replace each with Java's standard collection.

Investigate:

ArrayList
LinkedList
ArrayDeque
PriorityQueue
HashSet
LinkedHashSet
TreeSet
HashMap
LinkedHashMap
TreeMap

Understand **why there are so many variants**.

Java C10 — `Comparable` vs `Comparator`

Q201

Create:

Student

and sort by:

1. Roll number
2. Name
3. Marks

Implement:

Comparable<Student>

and separate:

Comparator<Student>

Then use:

Comparator.comparing()
thenComparing()
reversed()

Java C11 — Iterators

Q202

Iterate through:

ArrayList
HashSet
HashMap

using:

for
Iterator
enhanced for
forEach

Then investigate:

Iterator.remove()

and why directly modifying some collections during iteration can cause problems.

Java C12 — Lambdas

Q203

Revisit your C callback problem.

Implement:

processArray()

using Java:

Consumer
Predicate
Function
BiFunction

Then lambda expressions.

Now you can directly compare:

C function pointer
Java functional interface
Python function

TRACK C — PYTHON CONSOLIDATION

Python C1 — Classes

Q204

Create:

class Student:

with:

__init__()
display()
calculate_grade()

Then investigate:

self

and why Python explicitly has it.

Python C2 — Instance vs Class

Q205

Create:

instance variable
class variable
instance method
class method
static method

using:

@classmethod@staticmethod

Determine exactly when each should be used.

Python C3 — Properties

Q206

Create:

class BankAccount

with a private-ish balance.

Use:

@property

to control access.

Then implement:

@balance.setter

with validation.

Python C4 — Inheritance

Q207

Create:

Animal
 ├── Dog
 ├── Cat
 └── Bird

Implement:

speak()

and demonstrate polymorphism.

Python C5 — Multiple Inheritance

Q208

Create:

Person
Student

Employee
TeachingAssistant

where:

TeachingAssistant(Student, Employee)

Investigate:

__mro__

and:

super()

Python C6 — Abstract Base Classes

Q209

Investigate:

from abc import ABC, abstractmethod

Create:

Shape
Circle
Rectangle

with:

area()

Python C7 — Magic Methods

This becomes a major Python-specific exploration.

Q210

Create:

class Vector

and implement:

__str__
__repr__
__eq__
__add__
__sub__
__mul__
__len__

Then make:

Vector(2,3) + Vector(4,5)

produce:

Vector(6,8)

Investigate how Python lets objects participate naturally in language syntax.

Python C8 — Iterators

Q211

Create your own iterator:

class Counter:

implementing:

__iter__()
__next__()

so:

for x in Counter(5):

produces:

0 1 2 3 4

Python C9 — Generators

Q212

Rewrite the previous problem using:

yield

Compare:

iterator class
generator
list

in terms of:

- memory
- simplicity
- execution
- state

Python C10 — Context Managers

Q213

Create a custom context manager.

First understand:

with

and then implement:

__enter__()
__exit__()

Create something like:

```
with Timer():
    # some operation
```

that measures execution time.

Python C11 — Dataclasses

Q214

Create a `Student` using:

@dataclass

Compare it against a manually written class.

Investigate what the decorator generates for you.

Python C12 — Containers

Q215

Implement your own stack class.

Then investigate:

list
deque

and later:

collections.Counter
collections.defaultdict
collections.OrderedDict
heapq

Compare them against your own implementations.

FINAL CONSOLIDATION — SAME PROBLEM, THREE LANGUAGES

This is where everything starts coming together.

Q216 — Build a Student Management System

Create a small console application in **C, Java and Python**.

It should support:

1. Add student
2. Remove student
3. Search student
4. Update marks
5. Display all students
6. Sort students
7. Find topper
8. Find average
9. Find students above average
10. Save/load data

C

Use:

struct
dynamic memory
pointers
function pointers where useful
file handling

Java

Use:

classes
encapsulation
inheritance where justified
interfaces where useful
generics
ArrayList
HashMap
Comparator
exceptions
file handling

Python

Use:

classes
properties where useful
dataclasses
list/dict
functions as objects
sorting with key=

exceptions
file handling

Q217 — Same Architecture, Three Implementations

Now deliberately **don't translate the code line-by-line**.

Design the application according to each language's strengths.

Then answer:

C

> What did I have to manage manually?

Java

> What did the type system and OOD model give me?

Python

> What did dynamic typing, duck typing and built-in data structures simplify?

Q218 — The Ultimate Comparison Table

For each concept, fill this yourself:

Concept	C	Java	Python
Primitive integer	?	?	?
String	?	?	?
Array	?	?	?
Dynamic array	?	?	?
Pointer	?	?	?
Reference	?	?	?

Struct/class	?	?	?
Function pointer	?	?	?
Callback	?	?	?
Generic programming	?	?	?
Stack	?	?	?
Queue	?	?	?
Hash map	?	?	?
Set	?	?	?
Iterator	?	?	?
Exception	?	?	?
Memory management	?	?	?
OOD	?	?	?
File handling	?	?	?

Don't look up everything immediately. **Fill what you know first, then investigate what you don't.**

The important transition

Once you've completed this consolidation round, you'll have gone through:

Basic Programming
↓
Functions
↓
Strings
↓
Library/API exploration
↓
Arrays
↓
Searching
↓
Sorting
↓

Recursion
↓
Linked Lists
↓
Stacks
↓
Queues
↓
Hashing
↓
Trees
↓
Heaps
↓
Graphs
↓
———————————————————
Language Consolidation
———————————————————
↓
C → pointers → memory → function pointers
↓
Java → OOD → generics → collections → interfaces
↓
Python → OOD → duck typing → iterators → generators
↓
Same application in all 3

Only after this point should we open Part 2: OS-level programming.

And Part 2 can follow the exact same methodology:

OS basics → processes → system calls → IPC → files/processes → signals → threads → synchronization → deadlocks → scheduling → memory → C/Java/Python-specific OS capabilities → networking → sockets → concurrent servers → distributed programming.